

## Partie 2 : Des RNN aux Transformers

Bassem Ben Hamed

ENETCOM – Université de Sfax

Février 2026

## Section 2.1 — RNN et modèles séquentiels

- 1 Des représentations statiques aux contextuelles
- 2 Architecture RNN et états cachés
- 3 Gradients qui disparaissent/explosent
- 4 Architectures LSTM et GRU
- 5 RNN bidirectionnels
- 6 Seq2Seq avec attention
- 7 Limites des RNN

## Section 2.2 — La révolution Transformer

- 8 Le mécanisme d'attention
- 9 Self-attention : Query, Key, Value
- 10 Attention multi-têtes
- 11 Architecture du Transformer (Encodeur/Décodeur)
- 12 Encodages positionnels
- 13 Variantes BERT, GPT, T5
- 14 Tokenisation en sous-mots
- 15 Encodages positionnels modernes

À la fin de cette session, vous serez capable de :

- 1 **Comprendre** pourquoi les RNN étaient nécessaires et leurs limites fondamentales.
- 2 **Expliquer** les mécanismes LSTM/GRU et leurs améliorations par rapport aux RNN classiques.
- 3 **Maîtriser** le mécanisme d'attention et les calculs de self-attention.
- 4 **Décrire** l'architecture complète du Transformer.
- 5 **Distinguer** les modèles encodeur seul, décodeur seul et encodeur-décodeur.
- 6 **Comprendre** les stratégies modernes de tokenisation et d'encodage positionnel.
- 7 **Comparer** les caractéristiques de performance RNN vs Transformer.

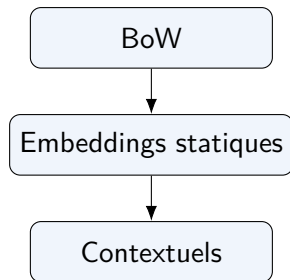
## Section 2.1

# Réseaux de neurones récurrents et leurs limites

Des modèles séquentiels à l'attention

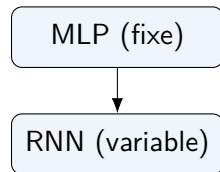
# Des représentations statiques aux contextuelles

- Word2Vec/GloVe = vecteurs statiques (même mot = même vecteur)
- Problème : *avocat* dans *avocat au tribunal* vs *salade d'avocat*
- Besoin : représentations sensibles au contexte
- Transition visuelle : BoW → embeddings statiques → embeddings contextuels



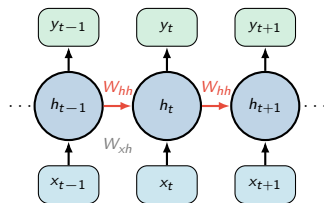
# Le besoin de modèles séquentiels

- Le langage est intrinsèquement séquentiel.
- Tâches nécessitant l'ordre : NER, traduction, sentiment avec contexte.
- Les FFN/MLP classiques échouent avec des séquences de longueur variable.
- Besoin de longueurs d'entrée/sortie variables.



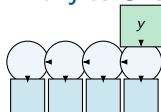
# Bases de l'architecture RNN

- L'état caché  $h_t$  porte la mémoire au fil du temps.
- Équation :  $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$
- Propagation avant dans le temps (déroulement).
- Partage des poids à travers les pas de temps.



# Applications des RNN : les 4 patterns

## Many-to-One



Sentiment, Classification

## One-to-Many

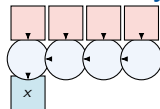
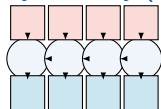


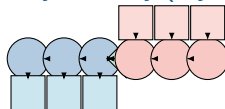
Image Captioning

## Many-to-Many (sync)



POS Tagging, NER

## Many-to-Many (async)

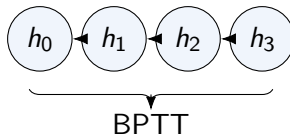


Traduction, Résumé



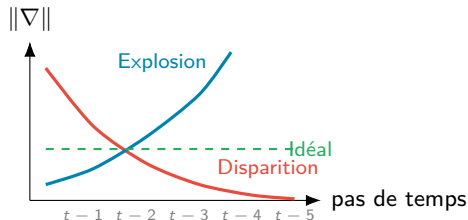
# Entraînement des RNN : rétropropagation dans le temps

- Déroulement du réseau à travers les pas de temps.
- Les gradients remontent à travers la séquence.
- Chaîne de dépendance :  $h_t \rightarrow h_{t+1}$ .
- Le graphe de calcul s'étend sur plusieurs pas.



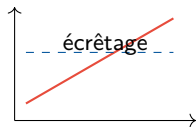
# Le problème du gradient qui disparaît

- Les gradients sont multipliés par  $W_{hh}$  à chaque pas.
- Si  $|W_{hh}| < 1$  : les gradients disparaissent, les dépendances à long terme sont perdues.
- Si  $|W_{hh}| > 1$  : les gradients explosent, l'entraînement est instable.
- Conséquence : difficulté à apprendre les dépendances à longue portée.



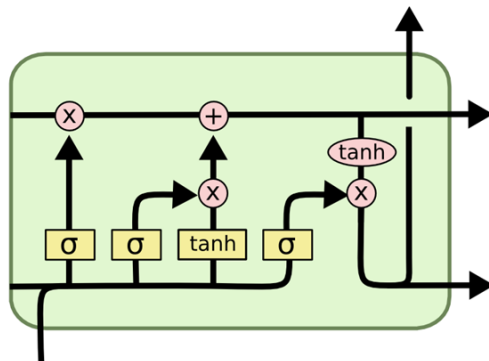
# Gradients explosifs et solutions

- Symptômes : poids NaN, la loss diverge.
- Solution 1 : gradient clipping.
- Solution 2 : régularisation des poids.
- Limitation : ne corrige pas la disparition des gradients.



# LSTM : Long Short-Term Memory

- **Motivation** : préserver l'information à long terme
- **État de cellule**  $c_t$  : l'autoroute de la mémoire
- **3 portes** contrôlent le flux d'information :
  - **Oubli** : effacer
  - **Entrée** : écrire
  - **Sortie** : lire



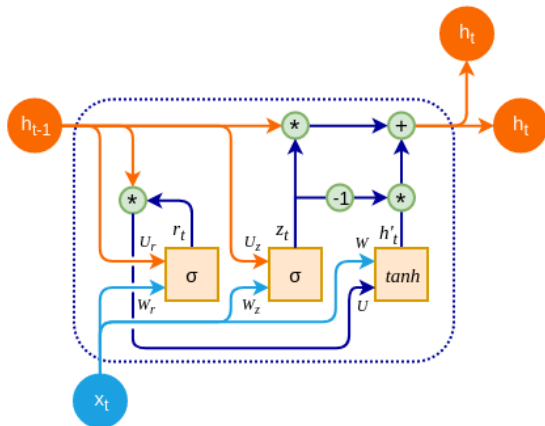


# GRU : Gated Recurrent Unit

- **Simplifie** le LSTM avec moins de paramètres
- **2 portes** : mise à jour ( $z_t$ ) et réinitialisation ( $r_t$ )
- Fusionne l'état de cellule et l'état caché
- Performance comparable, calcul réduit

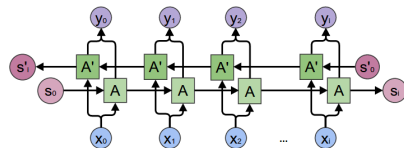
## Équations

$$\begin{aligned}z_t &= \sigma(W_z[h_{t-1}, x_t]) \\r_t &= \sigma(W_r[h_{t-1}, x_t]) \\\tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t]) \\h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t\end{aligned}$$



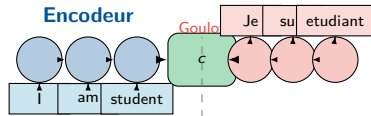
# RNN bidirectionnels

- Les RNN unidirectionnels ne voient que le passé.
- Le Bi-RNN utilise des passes avant et arrière.
- Concaténer les états cachés pour un contexte plus riche.
- Utile pour la NER, la classification, le QA.



# Séquence à séquence (Seq2Seq)

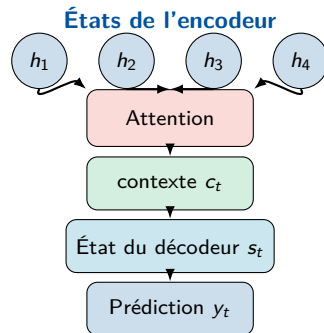
- Architecture **Encodeur-Décodeur**
- **Encodeur** : compresse la source en un vecteur de contexte
- **Décodeur** : génère la séquence cible
- **Goulot d'étranglement** : toute l'information dans un seul vecteur !





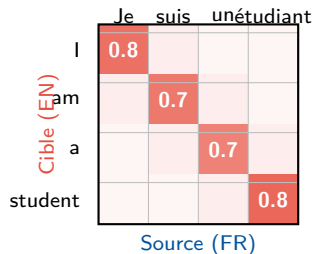
# Le mécanisme d'attention (Bahdanau 2015)

- Résout le goulot du vecteur de contexte.
- Le décodeur porte attention à tous les états de l'encodeur.
- Score d'alignement :  $score(s_t, h_i)$
- Poids :  $\alpha_{t,i} = \text{softmax}(score(s_t, h_i))$
- Contexte :  $c_t = \sum_i \alpha_{t,i} h_i$



# Visualisation des poids d'attention

- Carte thermique des poids d'attention.
- Exemple : alignement EN-FR.
- Interprétabilité : sur quoi le modèle se concentre-t-il ?
- Toujours séquentiel, donc lent pour les longues séquences.

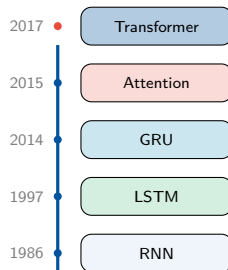


# Résumé : RNN et leurs limites

- RNN classique : simple mais gradients qui disparaissent.
- LSTM : portes pour la mémoire à long terme.
- GRU : LSTM simplifié.
- Bi-RNN : contexte bidirectionnel.
- Seq2Seq + Attention : mieux, toujours séquentiel.

## Limites principales :

- Calcul séquentiel (pas de parallélisme)
- Dépendances à longue portée difficiles
- Entraînement lent sur GPU



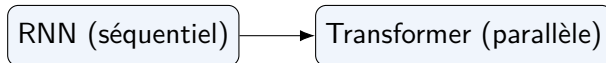
## Section 2.2

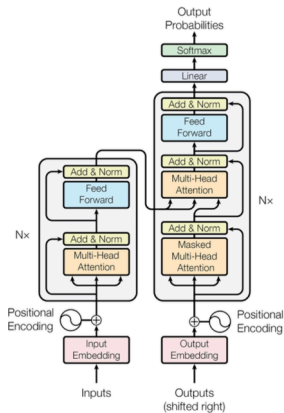
# La révolution Transformer

Attention Is All You Need

# Le problème des RNN (rappel)

- Calcul séquentiel  $\rightarrow$  lent.
- Pas de parallélisme sur les positions de la séquence.
- Difficulté avec les dépendances à longue portée (même LSTM).
- Besoin d'une architecture entièrement parallélisable.





## Le Transformer de A à Z

“Attention Is All You Need” — Vaswani et al., NeurIPS 2017

# Contexte : la tâche et les notations

## La tâche

Traduire **“je suis étudiant”** (3 tokens) en anglais.

## Vocabulaire partagé (10 mots)

| Index | Token    |
|-------|----------|
| 0     | <pad>    |
| 1     | <sos>    |
| 2     | <eos>    |
| 3     | je       |
| 4     | suis     |
| 5     | étudiant |
| 6     | I        |
| 7     | am       |
| 8     | a        |
| 9     | student  |

## Hyperparamètres simplifiés

| Paramètre          | Original | Exemple |
|--------------------|----------|---------|
| $d_{\text{model}}$ | 512      | 4       |
| $h$ (têtes)        | 8        | 2       |
| $d_k = d_v$        | 64       | 2       |
| $d_{\text{ff}}$    | 2048     | 8       |
| $N$ (couches)      | 6        | 1       |

## Séquences

**Entrée enc.** : [je, suis, étudiant]  $\rightarrow$  [3, 4, 5]

**Entrée déc.** : [<sos>]  $\rightarrow$  [1]

**Sortie** : [I, am, a, student, <eos>]

# Étape 0 — Embedding des tokens

## Formule

$$E(t) = W_{\text{emb}}[t] \times \sqrt{d_{\text{model}}}$$

- $W_{\text{emb}}$  : matrice  $|V| \times d_{\text{model}} = 10 \times 4$
- $\times \sqrt{d_{\text{model}}}$  : normalise à l'échelle des encodages positionnels
- $\sqrt{4} = 2$

$$X_{\text{emb}} = \begin{pmatrix} 1.0 & 0.6 & -0.4 & 0.2 \\ 0.4 & -0.2 & 0.8 & 0.6 \\ -0.6 & 1.2 & 0.2 & -0.8 \end{pmatrix} \begin{matrix} \leftarrow \text{je} \\ \leftarrow \text{suis} \\ \leftarrow \text{étudiant} \end{matrix}$$

## Calcul pour "je suis étudiant"

$$\begin{aligned} E(\text{je}) &= [0.5, 0.3, -0.2, 0.1] \times 2 \\ &= [1.0, 0.6, -0.4, 0.2] \end{aligned}$$

$$E(\text{suis}) = [0.4, -0.2, 0.8, 0.6]$$

$$E(\text{étud.}) = [-0.6, 1.2, 0.2, -0.8]$$



# Étape 1 — Encodage positionnel

## Pourquoi ?

Le Transformer n'a ni récurrence ni convolution. Sans position, “je suis étudiant” et “étudiant suis je” donnent la même représentation.

## Formules

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Dim. paires  $\rightarrow$  sin, impaires  $\rightarrow$  cos. Fréquences en progression géométrique de  $2\pi$  à  $10000 \cdot 2\pi$ .

## Calcul ( $d = 4, i \in \{0, 1\}$ )

### Fréquences :

$$i=0: \text{dén.} = 10000^0 = 1$$

$$i=1: \text{dén.} = 10000^{0.5} = 100$$

### Position 0 (je) :

$$\begin{aligned} & [\sin(0), \cos(0), \sin(0), \cos(0)] \\ & = [0.000, 1.000, 0.000, 1.000] \end{aligned}$$

### Position 1 (suis) :

$$\begin{aligned} & [\sin(1), \cos(1), \sin(0.01), \cos(0.01)] \\ & = [0.841, 0.540, 0.010, 1.000] \end{aligned}$$

### Position 2 (étudiant) :

$$\begin{aligned} & [\sin(2), \cos(2), \sin(0.02), \cos(0.02)] \\ & = [0.909, -0.416, 0.020, 1.000] \end{aligned}$$

## Étape 1 — Addition : Embedding + Position

### Formule

$$X = X_{\text{emb}} + PE$$

$$X(\text{je}) = [1.0+0.000, 0.6+1.000, -0.4+0.000, 0.2+1.000] = [\mathbf{1.000}, \mathbf{1.600}, \mathbf{-0.400}, \mathbf{1.200}]$$

$$X(\text{suis}) = [0.4+0.841, -0.2+0.540, 0.8+0.010, 0.6+1.000] = [\mathbf{1.241}, \mathbf{0.340}, \mathbf{0.810}, \mathbf{1.600}]$$

$$X(\text{étudiant}) = [-0.6+0.909, 1.2-0.416, 0.2+0.020, -0.8+1.000] = [\mathbf{0.309}, \mathbf{0.784}, \mathbf{0.220}, \mathbf{0.200}]$$

### Matrice $X$ finale (entrée de l'encodeur) — $3 \times 4$

$$X = \begin{pmatrix} 1.000 & 1.600 & -0.400 & 1.200 \\ 1.241 & 0.340 & 0.810 & 1.600 \\ 0.309 & 0.784 & 0.220 & 0.200 \end{pmatrix} \begin{array}{l} \leftarrow \text{je} \\ \leftarrow \text{suis} \\ \leftarrow \text{étudiant} \end{array}$$

## Étape 2 — L'Encodeur

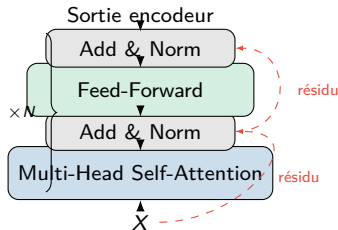
Chaque couche de l'encodeur contient **deux sous-couches** :

- 1 **Multi-Head Self-Attention**
- 2 **Position-wise Feed-Forward Network**

Chacune est enveloppée d'une **connexion résiduelle** et d'une **Layer Normalization** :

Formule générale

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$



# Scaled Dot-Product Attention

## Formule

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- $Q$  (Query) : “ce que je cherche”
- $K$  (Key) : “ce que je propose comme étiquette”
- $V$  (Value) : “ce que je contiens comme info”
- $\sqrt{d_k}$  : évite que les gradients du softmax ne deviennent trop petits

## Intuition : la bibliothèque

- $Q$  = votre question  
“un livre sur l'astronomie”
- $K$  = étiquettes des rayons  
“sciences”, “histoire”, “astronomie”
- $V$  = contenu des rayons
- softmax dit :  
astronomie 90%, sciences 8%, histoire 2%
- Sortie = mélange pondéré du contenu

# Attention : décomposition pas à pas

**Étape A — Projections linéaires :**  $Q_i = X \cdot W_i^Q (n \times d_k)$      $K_i = X \cdot W_i^K (n \times d_k)$   
 $V_i = X \cdot W_i^V (n \times d_v)$

**Étape B — Produit scalaire :**  $\text{scores} = Q_i \cdot K_i^T (n \times n)$  — cellule  $(i, j)$  : “à quel point le token  $i$  fait attention au token  $j$  ?”

**Étape C — Mise à l'échelle :**  $\text{scores\_scaled} = \text{scores} / \sqrt{d_k}$

**Étape D — Softmax (par ligne) :**  $\text{weights} = \text{softmax}(\text{scores\_scaled})$  — chaque ligne somme à 1 (distribution de proba sur les tokens)

**Étape E — Somme pondérée :**  $\text{head}_i = \text{weights} \cdot V_i$

# Multi-Head Attention

## Formule

$$\text{MH}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{Att}(XW_i^Q, XW_i^K, XW_i^V)$$

## Dimensions

$$W_i^Q \in \mathbb{R}^{4 \times 2} \quad W_i^K \in \mathbb{R}^{4 \times 2} \quad W_i^V \in \mathbb{R}^{4 \times 2}$$
$$W^O \in \mathbb{R}^{(h \cdot d_v) \times d_{\text{model}}} = 4 \times 4$$

$\text{head}_i$  = tête  $i$ , pas token  $i$

$\text{head}_i$  = sortie de la tête  $i$  pour **tous les tokens à la fois**.

## Pourquoi plusieurs têtes ?

Chaque tête apprend un **type de relation différent** :

- Tête 1 : relations syntaxiques (sujet-verbe)
- Tête 2 : relations sémantiques (coréférences)

Une seule tête ne voit pas ces relations.

# Calcul complet — Tête 1 : projections

—  $4 \times 2$  chacune :

$$W_1^Q = \begin{pmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \\ 0.4 & 0.0 \\ 0.0 & 0.2 \end{pmatrix} \quad W_1^K = \begin{pmatrix} 0.3 & -0.1 \\ 0.2 & 0.2 \\ -0.1 & 0.3 \\ 0.1 & 0.1 \end{pmatrix} \quad W_1^V = \begin{pmatrix} 0.1 & 0.4 \\ -0.2 & 0.1 \\ 0.3 & 0.0 \\ 0.0 & 0.2 \end{pmatrix}$$

**Projections**  $Q_1 = X \cdot W_1^Q$  ( $3 \times 4 \cdot 4 \times 2 = 3 \times 2$ ) :

$$Q_1(\text{je}) = [1.000 \times 0.2 + 1.600 \times (-0.1) + (-0.400) \times 0.4 + 1.200 \times 0.0, \dots] = [-0.120, 0.820]$$

**Résultats des projections** ( $3 \times 2$ ) :

$$Q_1 = \begin{pmatrix} -0.120 & 0.820 \\ 0.538 & 0.546 \\ 0.072 & 0.306 \end{pmatrix} \quad K_1 = \begin{pmatrix} 0.780 & 0.220 \\ 0.519 & 0.347 \\ 0.248 & 0.212 \end{pmatrix} \quad V_1 = \begin{pmatrix} -0.340 & 0.800 \\ 0.299 & 0.850 \\ -0.060 & 0.242 \end{pmatrix}$$

# Calcul complet — Tête 1 : scores et softmax

$$Q_1 \cdot K_1^T (3 \times 3) :$$

$$\text{scores}_1 = \begin{pmatrix} 0.087 & 0.222 & 0.144 \\ 0.540 & 0.469 & 0.250 \\ 0.124 & 0.144 & 0.083 \end{pmatrix}$$

(c) **Mise à l'échelle** :  $\div \sqrt{d_k} = \sqrt{2} \approx 1.414$  :

$$\text{scaled}_1 = \begin{pmatrix} 0.062 & 0.157 & 0.102 \\ 0.382 & 0.332 & 0.177 \\ 0.088 & 0.102 & 0.059 \end{pmatrix}$$

(d) **Softmax** (par ligne) :  $\text{softmax}(z_j) = e^{z_j} / \sum_k e^{z_k}$

$$W_1 = \begin{pmatrix} 0.318 & \mathbf{0.350} & 0.331 \\ \mathbf{0.361} & 0.344 & 0.295 \\ 0.335 & \mathbf{0.340} & 0.325 \end{pmatrix} \begin{array}{l} \leftarrow \text{ce que "je" regarde} \\ \leftarrow \text{ce que "suis" regarde} \\ \leftarrow \text{ce que "étudiant" regarde} \end{array}$$

*"je" porte 35.0% de son attention sur "suis", 33.1% sur "étudiant", 31.8% sur lui-même.*



# Calcul complet — Têtes 1 et 2 : résultats

$$\text{head}_1 = W_1 \cdot V_1 \quad (3 \times 3 \cdot 3 \times 2 = 3 \times 2)$$

$$\begin{aligned}\text{head}_1(\text{je}) &= 0.318 \times [-0.340, 0.800] + 0.350 \times [0.299, 0.850] + 0.331 \times [-0.060, 0.242] \\ &= [-0.023, 0.632]\end{aligned}$$

$$\text{head}_1 = \begin{pmatrix} -0.023 & 0.632 \\ -0.038 & 0.652 \\ -0.032 & 0.636 \end{pmatrix} \begin{array}{l} \leftarrow \text{je vu par tête 1} \\ \leftarrow \text{suis vu par tête 1} \\ \leftarrow \text{étud. vu par tête 1} \end{array} \quad \text{head}_2 = \begin{pmatrix} 0.150 & -0.100 \\ 0.220 & 0.050 \\ 0.180 & -0.030 \end{pmatrix} \begin{array}{l} \leftarrow \text{je vu par tête 2} \\ \leftarrow \text{suis vu par tête 2} \\ \leftarrow \text{étud. vu par tête 2} \end{array}$$

La tête 2 utilise des matrices  $W_2^Q$ ,  $W_2^K$ ,  $W_2^V$  **différentes**. Le calcul est identique.

# Concaténation et projection finale

— reconstituer chaque token en  $d_{\text{model}}$  :

je :  $[-0.023, 0.632] \parallel [0.150, -0.100] \rightarrow [-0.023, 0.632, 0.150, -0.100]$  (taille 4)  
suis :  $[-0.038, 0.652] \parallel [0.220, 0.050] \rightarrow [-0.038, 0.652, 0.220, 0.050]$   
étudiant :  $[-0.032, 0.636] \parallel [0.180, -0.030] \rightarrow [-0.032, 0.636, 0.180, -0.030]$

**Projection par  $W^O$  ( $4 \times 4$ ) :**

$$\text{MH\_out} = \text{Concat}(\text{head}_1, \text{head}_2) \cdot W^O = \begin{pmatrix} 0.210 & 0.450 & -0.180 & 0.320 \\ 0.280 & 0.510 & 0.040 & 0.390 \\ 0.230 & 0.470 & -0.090 & 0.340 \end{pmatrix}$$

Le  $i$  dans  $\text{head}_i$

= numéro de la **tête**, pas du token. Chaque tête traite **tous** les tokens.

# Add & Norm : connexion résiduelle + LayerNorm

## Connexion résiduelle

$$Z = X + \text{MH\_out}$$

- Évite la disparition du gradient (comme ResNet)
- L'information d'origine n'est jamais perdue

## Layer Normalization

$$\text{LN}(z) = \gamma \odot \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$$\mu = \frac{1}{d} \sum_j z_j \quad \sigma^2 = \frac{1}{d} \sum_j (z_j - \mu)^2$$

## Exemple : token "je"

$$\begin{aligned} Z(\text{je}) &= X(\text{je}) + \text{MH\_out}(\text{je}) \\ &= [1.210, 2.050, -0.580, 1.520] \end{aligned}$$

$$\begin{aligned} \mu &= (1.210 + 2.050 - 0.580 + 1.520)/4 \\ &= 1.050 \end{aligned}$$

$$\sigma^2 = 0.976 \quad \sigma = 0.988$$

$$\begin{aligned} \text{LN}(\text{je}) &= \left[ \frac{1.210 - 1.050}{0.988}, \dots \right] \\ &= [0.162, 1.012, -1.650, 0.476] \end{aligned}$$

# Feed-Forward Network (FFN)

## Formule

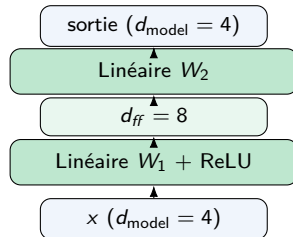
$$\text{FFN}(x) = \max(0, xW_1 + b_1) \cdot W_2 + b_2$$

Deux couches avec activation **ReLU** :

- Couche cachée :  $h = \text{ReLU}(xW_1 + b_1)$   
 $d_{\text{model}} \rightarrow d_{\text{ff}} \quad (4 \rightarrow 8)$
- Sortie :  $o = hW_2 + b_2$   
 $d_{\text{ff}} \rightarrow d_{\text{model}} \quad (8 \rightarrow 4)$

## Points importants

- Appliqué **indépendamment** à chaque position
- Équivalent à deux convolutions  $1 \times 1$
- $d_{\text{ff}} = 4 \times d_{\text{model}}$  dans le papier



## Deuxième Add & Norm

$$\text{Encoder\_out} = \text{LayerNorm}(Z_{\text{norm}} + \text{FFN}(Z_{\text{norm}}))$$

**Résultat final de l'encodeur** ( $3 \times 4$ ) — noté **Enc** :

$$\mathbf{Enc} = \begin{pmatrix} \text{enc}_{11} & \text{enc}_{12} & \text{enc}_{13} & \text{enc}_{14} \\ \text{enc}_{21} & \text{enc}_{22} & \text{enc}_{23} & \text{enc}_{24} \\ \text{enc}_{31} & \text{enc}_{32} & \text{enc}_{33} & \text{enc}_{34} \end{pmatrix} \quad \begin{array}{l} \leftarrow \text{repr. contextuelle de "je"} \\ \leftarrow \text{repr. contextuelle de "suis"} \\ \leftarrow \text{repr. contextuelle de "étudiant"} \end{array}$$

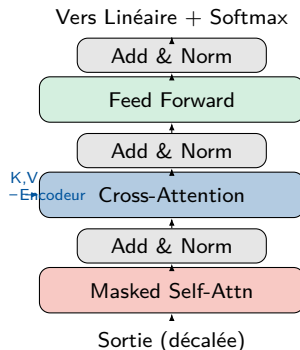
## Point clé

Ces vecteurs ne représentent plus de simples mots isolés. Grâce à la self-attention, chaque vecteur contient de l'information sur **toute la phrase**.

## Étape 3 — Le Décodeur

Le décodeur a **trois sous-couches** (au lieu de deux) :

- 1 **Masked Multi-Head Self-Attention** — ne regarde que les tokens passés
- 2 **Multi-Head Cross-Attention** —  $Q$ =décodeur,  $K, V$ =encodeur
- 3 **Feed-Forward Network**



Chacune est suivie d'un Add & Norm.

# Masked Self-Attention : pourquoi le masque ?

Le décodeur génère les tokens **un par un** (auto-régressif). À la position  $t$ , il ne doit **pas voir les positions**  $> t$ .

## Formule

$$\text{Attn} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask}\right) V$$

Le masque est une matrice triangulaire supérieure de  $-\infty$  :

$$\text{Mask} = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Après softmax :  $e^{-\infty} = 0 \rightarrow$  positions ignorées.

## Exemple au pas $t = 2$

Entrée déc. : [ $\langle \text{sos} \rangle$ , I]

$$\text{scores} + \text{Mask} = \begin{pmatrix} s_{11} & -\infty \\ s_{21} & s_{22} \end{pmatrix}$$

$$\xrightarrow{\text{softmax}} \begin{pmatrix} 1.000 & 0.000 \\ 0.450 & 0.550 \end{pmatrix}$$

"I" peut voir  $\langle \text{sos} \rangle$  et lui-même, mais  $\langle \text{sos} \rangle$  ne se voit que lui-même.

# Cross-Attention (attention encodeur-décodeur)

C'est la **passerelle** entre l'encodeur et le décodeur.

## Formule

$$Q = \text{sortie\_masked} \cdot W^Q \leftarrow \text{décodeur}$$

$$K = \text{Enc} \cdot W^K \leftarrow \text{encodeur}$$

$$V = \text{Enc} \cdot W^V \leftarrow \text{encodeur}$$

**Pas de masque** ici : le décodeur voit **toute** la phrase source.

## Exemple au pas $t = 2$

$QK^T$  :  $(2 \times 3)$  — chaque token du décodeur assigne un score à chaque token source.

|              | "je" | "suis" | "étud."     |
|--------------|------|--------|-------------|
| scores(I, *) | 0.8  | 0.3    | 1.2         |
| softmax      | 0.30 | 0.15   | <b>0.55</b> |

*Pour prédire après "I", le modèle regarde surtout "étudiant" (55%) et "je" (30%).*



## Étape 4 — Projection linéaire + Softmax

On prend le **dernier vecteur** de la sortie du décodeur et on le projette sur le vocabulaire :

### Formules

$$\text{logits} = \text{Dec\_out}[-1] \cdot W_{\text{vocab}} + b$$

$$P(\text{mot}_i) = \frac{e^{\text{logit}_i}}{\sum_j e^{\text{logit}_j}}$$

$$W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times |V|} = 4 \times 10$$

Note :  $W_{\text{vocab}} = W_{\text{emb}}^T$  (poids partagés).

Pas  $t = 1$  : prédire après <sos>

| Token                    | $e^{\text{logit}}$ | $P$         |
|--------------------------|--------------------|-------------|
| pad                      | 0.12               | .002        |
| sos                      | 0.01               | .000        |
| eos                      | 0.04               | .001        |
| je                       | 0.17               | .003        |
| suis                     | 0.01               | .000        |
| étud.                    | 0.02               | .000        |
| <b>I</b>                 | <b>44.70</b>       | <b>.878</b> |
| am                       | 1.65               | .032        |
| a                        | 1.22               | .024        |
| student                  | 3.00               | .059        |
| Mot prédit : "I" (87.8%) |                    |             |

## Étape 5 — Génération auto-régressive

Le décodeur génère la traduction **mot par mot**. Le mot prédit est ajouté à l'entrée :

| Pas | Entrée décodeur            | Prédit    |     |
|-----|----------------------------|-----------|-----|
| 1   | [<sos>]                    | "I"       | ✓   |
| 2   | [<sos>, I]                 | "am"      | ✓   |
| 3   | [<sos>, I, am]             | "a"       | ✓   |
| 4   | [<sos>, I, am, a]          | "student" | ✓   |
| 5   | [<sos>, I, am, a, student] | "<eos>"   | FIN |

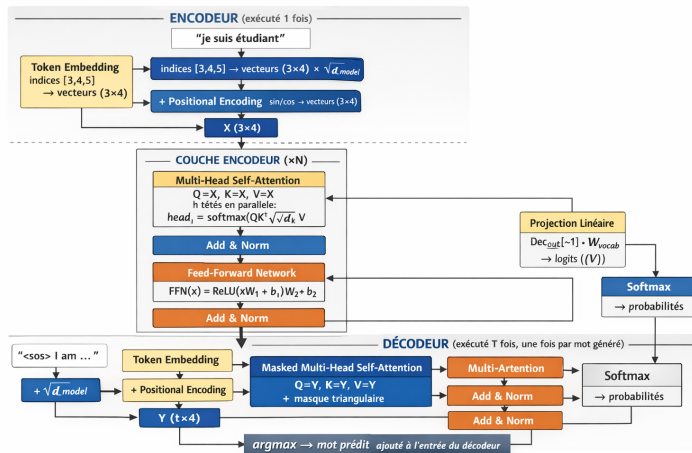
### Point clé

L'encodeur ne s'exécute **qu'une seule fois**. Sa sortie **Enc** est réutilisée à chaque pas du décodeur (via la cross-attention). Seul le décodeur est ré-exécuté avec une séquence d'entrée qui grandit.

# Récapitulatif de toutes les formules

| # | Composant        | Formule                                                                                                              | Rôle                                  |
|---|------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| 1 | Embedding        | $E(t) = W_{\text{emb}}[t] \times \sqrt{d_{\text{model}}}$                                                            | Indices $\rightarrow$ vecteurs denses |
| 2 | Enc. positionnel | $PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d})$<br>$PE_{(\text{pos}, 2i+1)} = \cos(\text{pos}/10000^{2i/d})$ | Injecter la position                  |
| 3 | Scaled Attention | $\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$                                          | Mélanger l'info entre tokens          |
| 4 | Multi-Head       | $\text{MH} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$                                                  | Relations variées                     |
| 5 | FFN              | $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$                                                                       | Transformation non-linéaire           |
| 6 | Add & Norm       | $\text{LayerNorm}(x + \text{Sublayer}(x))$                                                                           | Stabiliser l'entraînement             |
| 7 | Masque           | scores + Mask ( $-\infty$ pour le futur)                                                                             | Causalité au décodeur                 |
| 8 | Sortie           | $P(\text{mot}) = \text{softmax}(\text{Dec} \cdot W_{\text{vocab}})$                                                  | Distribution sur le vocabulaire       |
| 9 | Learning rate    | $lr = d^{-0.5} \cdot \min(s^{-0.5}, s \cdot w^{-1.5})$                                                               | Warm-up puis décroissance             |

# Récapitulatif : flux complet des données



L'encodeur traite la phrase source une seule fois, puis le décodeur génère la traduction token par token via la cross-attention.